Volve Assistant — Spec

Personal briefing + open-loop tracker for one user (Fink), running on Oracle VM, interfaced via Slack.

Status: design, pre-build. Target: Phase 0 live in 1 week, full system in 3–4 weeks of part-time work.

Goals

The system must:

1. **Not let me miss things.** Unreplied @-mentions (Linear, Notion, Slack), unanswered emails from watched senders, meetings without prep, assigned issues going stale. Detection is deterministic — a guarantee, not a best-effort.
2. **Give context on demand.** I DM the assistant in Slack with an open question; it answers using live data from Linear/Notion/Gmail/Calendar/Slack/GitHub plus my hand-written context.
3. **Push scheduled digests.** Morning (08:00) day-at-a-glance. Evening (18:00) wrap + still-open loops. Both to a dedicated Slack DM.
4. **Push event alerts.** Meeting in 30 min with no prep → contextualized nudge. P0/P1 Linear issue assigned → alert with history. Email from exec/key customer → alert with thread context.
5. **Feel like it knows me.** Knows my role, who Volve's people are, what I'm working on, my tone preferences. Updates as context drifts.

The system must NOT (for now):

- Act on my behalf (send emails, resolve Linear tickets, edit Notion). Read-only first, write later only where explicit.
- Serve anyone other than me. Single-user by design.
- Replace pi or interfere with `sv`/solve-ticket. Separate artifact, separate purpose.
- Be a chat companion. Task-scoped. Short turns. No personality bloat.

Architecture

Three layers, intentionally split by determinism:

```
┌──────────────────────────────────────────────────────────┐
│  LAYER 1: WATCHERS (deterministic, cron-driven)          │
│  Python scripts, one per source. Poll → diff → write events │
│  to SQLite. No LLM. This is what guarantees nothing is   │
│  missed.                                                 │
└──────────────────────────────────────────────────────────┘
                            │
                            ▼
┌──────────────────────────────────────────────────────────┐
│  LAYER 2: STATE (SQLite on VM)                           │
│  Tables: events, open_loops, entities, sessions, context. │
│  Ground truth for "what's unresolved".                   │
└──────────────────────────────────────────────────────────┘
                            │
                            ▼
┌──────────────────────────────────────────────────────────┐
│  LAYER 3: AGENT (Claude Code headless, invoked per-event) │
│  Reads state DB + VOLVE.md + session log. MCP for live   │
│  follow-up queries. Writes to Slack.                     │
│  Triggers: cron (digests) + Slack DM (on-demand) +       │
│  watcher events above urgency threshold.                 │
└──────────────────────────────────────────────────────────┘
                            │
                            ▼
┌──────────────────────────────────────────────────────────┐
│  SLACK (I/O)                                             │
│  Socket Mode bot (WebSocket out, no public ingress).     │
│  Inbound: DMs → agent. Outbound: agent → DM channel.     │
│  Simple alerts from watchers bypass LLM when possible.   │
└──────────────────────────────────────────────────────────┘
```

Why this split. The "don't miss things" guarantee is a correctness property. Correctness properties need deterministic code. An LLM polling for "has anything changed?" is expensive, nondeterministic, and slow. The LLM's job is interpretation and framing, not detection.

Runtime choice — Claude Code (not pi). Claude Code is included with every Volve Team seat at zero marginal cost. Native MCP support covers every source we need with official or mature third-party servers. Pi has excellent dev-workflow skills (`sv`, `solve-ticket`) but building MCP-equivalent extensions for Gmail, Calendar, Slack is rebuild work with no upside. Pi stays the coding tool. Claude Code runs the assistant.

Dev tool choice. Pi on Mac with Opus 4.7 via Copilot is fine for building this. No change to your workflow.

Component: Watchers

One process per source. Python 3.11+. Run via systemd timer (preferred over cron — better logging, dependency handling, failure recovery).

Watcher contract

Each watcher:

1. Reads its last-run cursor from `state.sqlite`.
2. Fetches changes since cursor from source API.
3. Writes new raw events to `events` table.
4. Applies promotion rules → writes/updates/closes rows in `open_loops`.
5. Updates cursor.

Idempotent. A watcher that runs twice produces the same state. A watcher that crashes mid-run is safe to re-run.

Watchers to build (priority order)

#	Source	Polls	Detects	Promotes to open_loop when
1	Linear	every 5 min	issue assigned to me, comment @me, status change on my issues	comment @me, no reply from me after 4h; issue assigned to me idle ≥3 days
2	Calendar	every 15 min	upcoming meetings (next 24h), declined meetings	meeting starts in ≤30 min and no attached prep doc
3	Gmail	every 5 min	threads where I'm recipient, threads from watched senders	watched-sender email unread ≥2h; thread where I was last addressed, no reply ≥24h
4	Notion	every 10 min	comments @me, pages I own with new comments	comment @me unresolved ≥4h
5	Slack	every 2 min	DMs to me, @-mentions of me in joined channels	@me or DM, no reaction/reply from me ≥2h during work hours
6	GitHub	every 10 min	PRs requesting my review, comments on my PRs	review requested ≥4h; comment on my PR ≥8h

Thresholds above are starting values. Expect to tune them once you have real signal. Store thresholds in config, not code.

Watcher implementation notes

- **Gmail watcher** needs a watchlist of senders (exec, key customers) — maintained in a

YAML file. Mass filter: ignore everything not from the watchlist for alerting purposes; still index for querying.

- **Calendar watcher** must resolve "has prep doc" heuristically — look for a Notion URL in event description or attached. False negatives are OK (user can always say "I'm prepped"); false positives (missed prep check) are the failure mode we care about.
- **Slack watcher** needs to know your working hours (from VOLVE.md) — no alerts at 23:00 for a message posted at 22:50.
- **Linear watcher** is the highest-value one. Start here. You already have `extensions/linear.ts` for pi — reuse the auth pattern.

Component: State (SQLite)

One file, `~/volve/state.sqlite`. WAL mode. Backed up nightly to the VM's `/home/jskontorp/backups/`.

Schema

```sql
```

```sql
-- Every observation a watcher makes
CREATE TABLE events (
  id INTEGER PRIMARY KEY,
  source TEXT NOT NULL,           -- 'linear', 'notion', 'gmail', ...
  source_id TEXT NOT NULL,        -- external ID (issue ID, thread ID, etc.)
  kind TEXT NOT NULL,             -- 'comment_mention', 'assigned', 'status_change',
  observed_at INTEGER NOT NULL,   -- unix timestamp
  occurred_at INTEGER,            -- when the thing actually happened, if known
  payload_json TEXT NOT NULL,     -- full source-specific data
  UNIQUE(source, source_id, kind, occurred_at)
);
CREATE INDEX idx_events_observed ON events(observed_at);

-- Things that need my attention and aren't yet resolved
CREATE TABLE open_loops (
  id INTEGER PRIMARY KEY,
  source TEXT NOT NULL,
  source_id TEXT NOT NULL,
  kind TEXT NOT NULL,             -- 'unreplied_mention', 'stale_assignment', ...
  opened_at INTEGER NOT NULL,
  last_nudged_at INTEGER,         -- when we last told user about it
  urgency REAL NOT NULL,          -- 0.0-1.0, grows with time
  title TEXT NOT NULL,            -- human-readable one-liner
  deep_link TEXT,                 -- URL back to the source
  context_json TEXT,              -- what else to say when surfacing
  resolved_at INTEGER,            -- NULL = still open
  UNIQUE(source, source_id, kind)
);
CREATE INDEX idx_open_urgency ON open_loops(urgency DESC) WHERE resolved_at IS NULL;

-- Known people/companies/projects, enriched over time
CREATE TABLE entities (
  id INTEGER PRIMARY KEY,
  kind TEXT NOT NULL,             -- 'person', 'company', 'project'
  canonical_name TEXT NOT NULL,
  aliases_json TEXT,              -- ["Anna", "anna.olsen@volve.no", "U038..."]
  notes TEXT,                     -- free-form, edited by user or agent
  UNIQUE(kind, canonical_name)
);

-- Agent invocations, for "we discussed X on Tuesday" recall
CREATE TABLE sessions (
  id INTEGER PRIMARY KEY,
  started_at INTEGER NOT NULL,
  trigger TEXT NOT NULL,          -- 'morning_digest', 'slack_dm', 'event_meeting_so
  prompt TEXT,
```

```
  response TEXT,
  tool_calls_json TEXT
);
CREATE INDEX idx_sessions_started ON sessions(started_at DESC);
```

Why SQLite and not Postgres. One user, one host, no concurrent writers beyond the few watchers. SQLite with WAL handles this trivially. Postgres is operational overhead with no benefit here.

Urgency function

```
urgency = min(1.0, age_hours / threshold_hours) * kind_weight
```

Where `kind_weight` reflects inherent severity (exec email 0.9, stale assignment 0.4, etc.). Pure function of (kind, age). Computed on read, not stored. Rewrite if this turns out to be too crude.

Component: Agent

Claude Code invoked headlessly. Three invocation modes:

Mode A — Scheduled digest

```bash
claude -p "$(cat prompts/morning_digest.md)" \
  --context-file ~/volve/VOLVE.md \
  --context-file <(scripts/state_summary.py --mode morning) \
  --mcp-config ~/.config/claude/mcp.json \
  --output-format text \
  | scripts/post_to_slack.py --channel $VOLVE_DM_CHANNEL
```

Runs at 08:00 and 18:00 via systemd timer. `state_summary.py` dumps current open loops + today's calendar + recent activity into a concise text blob. VOLVE.md provides stable context.

Mode B — Slack DM (on-demand)

Bolt-for-Python bot receives DM → drops message onto a Redis queue (or just a filesystem queue for MVP) → worker pops it and invokes Claude Code with the message as prompt + state summary + VOLVE.md → response posts back to the same thread. Latency target: p50 under 15s, p95 under 45s.

Mode C — Event-triggered

When a watcher promotes an event above urgency 0.8 and `last_nudged_at` is stale:

- If the event is simple (meeting in 30 min, no prep): direct Slack post from watcher, no LLM.

- If the event needs framing (exec email with thread history, P0 assignment with context): invoke Claude Code with a targeted prompt ("contextualize this alert: $EVENT"), post result.

This split keeps cost and latency down for routine alerts while using the LLM where it actually adds value.

MCP config

`~/.config/claude/mcp.json` connects:

- **Linear** — official MCP server, OAuth to Volve workspace
- **Notion** — official MCP server, OAuth to Volve workspace
- **Gmail** — official or `google-workspace` MCP server, OAuth (service account if available)
- **Google Calendar** — same as Gmail
- **Slack** — official MCP server for reads (user-scoped)
- **GitHub** — official MCP server, PAT or OAuth

Auth in a daemon context is the known-risky piece — see Open Questions.

VOLVE.md — the "it knows me" lever

Hand-maintained by you, ~2-5 KB. Read into context on every invocation. Template:

```markdown
```

```
# Volve context

## Me
- Name: Fink (Jan Sigurd Kontorp)
- Role: [role]
- Working hours: 08:00-17:00 Europe/Oslo, Mon-Fri
- Tone: direct, no filler, flag conflicts, don't pad uncertainty

## Company
- What Volve does: [one paragraph]
- Stage: [seed/series A/etc], headcount: [N]

## People
- [Name] — [role] — [how I relate to them, e.g. "my manager", "co-founder"]
- ...

## Current focus
- [Project] — [my role, current state, next milestone]
- ...

## Standing commitments
- Weekly 1:1 with [manager], Tuesdays 10:00
- ...

## Watched senders (for Gmail alerting)
- [email addresses] — [why]

## Don't alert me about
- [anti-patterns, e.g. "automated newsletters", "CI notifications"]
```

The file is truth. If the agent gets someone's role wrong, you edit VOLVE.md and the next invocation has it right. No model fine-tuning, no vector DB — just a markdown file. This is the *simplest thing that works* and shouldn't grow until it proves insufficient.

Component: Slack interface

Bolt-for-Python, Socket Mode. Single Slack app installed in Volve workspace.

Scopes needed (bot):

- `chat:write` — post messages
- `im:history` — read DMs to the bot
- `im:read` — see DM channel
- `users:read` — resolve user IDs

Scopes needed (user, for the Slack watcher):

- `search:read` — find @-mentions of me
- `channels:history`, `groups:history`, `im:history`, `mpim:history` — read from joined channels

Socket Mode means no public ingress required on the Oracle VM. WebSocket out from the bot process to Slack. Matches your Tailscale-only posture.

UX:

- One DM channel between me and the bot. All interaction happens there.
- Free-form messages. No slash commands until they prove necessary.
- Proactive messages appear in the same DM, visually distinct ("🔔 Nudge:" prefix or similar).
- A reaction on a bot message (✅) resolves the associated open loop. Small but valuable closing-the-loop primitive.

Repo layout

Assuming the separate-repo pattern (flag if you want otherwise):

```
~/code/work/volve-assistant/           # NEW repo — the daemon
├── README.md
├── pyproject.toml
├── volve/
│   ├── __init__.py
│   ├── db.py                          # SQLite connection + migrations
│   ├── watchers/
│   │   ├── base.py                    # shared Watcher base class
│   │   ├── linear.py
│   │   ├── calendar.py
│   │   ├── gmail.py
│   │   ├── notion.py
│   │   ├── slack.py
│   │   └── github.py
│   ├── agent/
│   │   ├── invoke.py                  # subprocess wrapper around claude -p
│   │   ├── state_summary.py           # DB → text blob for context
│   │   └── prompts/
│   │       ├── morning_digest.md
│   │       ├── evening_wrap.md
│   │       └── event_contextualize.md
│   ├── slack_bot/
│   │   ├── main.py                    # Bolt app
│   │   └── handlers.py
│   └── cli.py                         # `volve watch linear`, `volve digest`,
etc.
├── config/
│   ├── watchlist.yaml                 # Gmail senders to track
│   ├── thresholds.yaml                # promotion thresholds per kind
│   └── mcp.json.example
├── deploy/
│   ├── systemd/
│   │   ├── volve-watcher-linear.service
│   │   ├── volve-watcher-linear.timer
│   │   ├── ...
│   │   ├── volve-slack-bot.service
│   │   └── volve-digest.timer
│   └── install.sh
├── VOLVE.md                           # the context file, gitignored or encrypted
└── tests/

~/dotfiles/projects/volve-assistant/   # pi config for working on this repo
├── AGENTS.md                          # pi-specific coding rules for this project
├── skills/                            # any project-specific skills
```

```
└── extensions/                          # if you want pi-side helpers while
building
```

`projects.conf` gets a new line:

```
volve-assistant  ~/code/work/volve-assistant
```

Nothing else in dotfiles needs changing.

Auth & secrets

- `~/.env.volve` on the VM (mode 600). Contains: Anthropic-side Claude Code auth (via `claude login` on first setup), Slack bot token, Slack app token, Linear API key or OAuth token, Notion integration token, Google OAuth client + refresh token, GitHub PAT.
- Systemd units load it via `EnvironmentFile=`.
- Nothing committed. VOLVE.md either gitignored or encrypted with `git-crypt` / `age`.
- First-time OAuth flows handled via SSH port-forward from Mac: run the OAuth initiator on the VM, tunnel the callback port to your Mac, complete the flow in your Mac browser, refresh token lands on the VM. Standard pattern, one-time cost per source.

Deployment

Oracle VM, existing. Supervised by systemd.

Unit	Type	Cadence / Trigger
`volve-watcher-linear.timer`	timer	every 5 min
`volve-watcher-calendar.timer`	timer	every 15 min
`volve-watcher-gmail.timer`	timer	every 5 min
`volve-watcher-notion.timer`	timer	every 10 min
`volve-watcher-slack.timer`	timer	every 2 min
`volve-watcher-github.timer`	timer	every 10 min
`volve-digest-morning.timer`	timer	08:00 Mon–Fri
`volve-digest-evening.timer`	timer	18:00 Mon–Fri
`volve-slack-bot.service`	long-running	systemd auto-restart

Logs via `journalctl -u volve-*`. No separate log aggregation until volume justifies it.

Deploy: `cd ~/code/work/volve-assistant && git pull && just install`. The `justfile` runs `pip install -e .`, copies systemd units, reloads, restarts.

Build plan

Four phases, each independently useful. Don't build N+1 until N works in daily use.

Phase 0 — Cron-only digest (target: 1 week, part-time)

Simplest thing that could work. No watchers. No bot. Just:

- SQLite file created, schema applied
- One systemd timer at 08:00 invokes Claude Code headless
- Prompt: "Produce a morning brief. Use MCP to fetch: today's calendar, Linear issues assigned to me, unread Gmail from [watchlist], Notion comments @me from last 24h. Format: sections with bullets, under 200 words total, lead with what's most time-sensitive."
- Output posted to Slack DM via webhook (no bot yet — incoming webhook is 10 min of setup)
- VOLVE.md exists, minimal (me + people + current focus)

Success criterion: for 5 weekdays in a row, the 08:00 message tells you something useful you'd have otherwise had to find yourself. If it fails this test, the whole premise needs rethinking before adding complexity.

Phase 1 — Slack bot for pull queries (target: +3 days)

- Slack app created, Bolt-for-Python service running
- DM → invocation → response works
- Same MCP config as Phase 0
- Still no watchers — agent fetches everything live per query

Success criterion: 3 weekdays where at least one DM query saves you real time (e.g., "what's the state of [project]?" surfaces something faster than clicking through Linear + Notion yourself).

Phase 2 — Watchers + open_loops (target: +1 week, iterative)

Build watchers one source at a time, in priority order (Linear first). Each watcher ships independently:

- Linear watcher live → open_loops populated → morning digest includes them
- Calendar watcher live → meeting-prep detection added to digest
- Continue through Gmail, Notion, Slack, GitHub as each proves useful or you hit a miss the current setup wouldn't have caught

Success criterion per watcher: within 5 days of shipping, the watcher catches ≥1 thing you'd have otherwise missed, with ≤2 false positives per week.

Phase 3 — Event-triggered alerts (target: +3 days)

- Urgency threshold + last_nudged_at logic wired
- Simple alerts (meeting in 30 min) post directly
- Complex alerts (exec email, P0 assign) route through Claude Code for framing
- Reaction-to-resolve wired (✅ on a nudge marks the open_loop resolved)

Success criterion: one week where proactive alerts fire ≥3 times and each time the alert was warranted (you'd have wanted to know).

Open questions (resolve during build, not now)

1. **Headless Claude Code auth on a daemon VM.** `claude login` is OAuth-interactive. Token persists after, but I don't know the refresh behavior in a long-running unattended context. Mitigation: run through it on Phase 0, fall back to `ANTHROPIC_API_KEY` (pay-per-token) if subscription auth is unworkable headless. Rough cost guardrail if we fall back: Opus 4.6 at $5/$25 per MTok, ~10 invocations/day at ~50k tokens each = ~$2/day. Tolerable but not free.
2. **Official MCP servers vs local.** Anthropic-hosted MCP connectors may or may not support non-interactive auth refresh cleanly in 2026. Fallback: run local open-source MCP servers

with service-account creds. Test on Phase 0 with Linear + Calendar; decide before Phase 2.

3. **Notion MCP coverage of comments.** Need to verify the MCP server surfaces comment-level data with @-mention detail, not just page content. If not, the Notion watcher hits Notion's REST API directly.
4. **Slack user-token vs bot-token for reads.** Bot tokens can't see DMs not addressed to the bot. Watching your own @-mentions across channels requires a user token. Slack's UX for user-token installs is friction-ier than bot-only; confirm this is acceptable before Phase 2 (the Slack watcher).
5. **VOLVE.md drift.** The file needs to stay current. Options: (a) discipline — you edit it weekly; (b) agent suggests updates at end of evening wrap ("I noticed you now work with X, want me to append?"); (c) both. Decide after Phase 1 when you have a feel for drift rate.

Not building (scope guard)

Explicitly out of scope until justified:

- Vector DB / RAG for past sessions. SQLite + recent-sessions dump in context is enough for one user.
- Multi-user support. Single user by design; adding users means rethinking auth and permissioning.
- Agent takes actions (replies to emails, closes issues). Read-only until you trust the system end-to-end — probably months.
- Voice I/O. Text in Slack is fine.
- Web dashboard. Slack is the UI. If you want a "list of open loops" view, a single `volve open` CLI command is cheaper than a web UI.
- Learning loop / fine-tuning on your preferences. VOLVE.md is the learning loop.

Risks, mitigations

Risk	Likelihood	Impact	Mitigation
Notification fatigue (bot cries wolf)	High	High	Start with conservative thresholds, tune up only when quiet; require reaction-to-resolve so patterns of ignored alerts are visible.
Watcher misses something (false negative)	Medium	High	Each watcher has a "why didn't you tell me about X?" manual-test checklist. If something gets missed, add a test case and fix the watcher.

Risk	Likelihood	Impact	Mitigation
OAuth token expiry crashes a watcher silently	Medium	Medium	All watchers emit heartbeat to DB; a heartbeat watcher alerts if any source goes ≥2 intervals without reporting.
Claude Code subscription usage cap hit	Low-medium	Medium	Monitor token usage in session logs; Phase 0 has one invocation per day. Don't escalate frequency until signal justifies it.
VOLVE.md becomes stale, advice degrades	Medium	Medium	Weekly calendar recurring event: "15 min — review VOLVE.md".
Over-engineering	High (self-warning)	High	Phase gates are actual gates. Don't start Phase 2 until Phase 1 has run for a week in daily use.

First concrete step

```bash
mkdir -p ~/code/work/volve-assistant && cd $_
git init
pnpm-style bootstrap via pi: open pi, prompt it with this spec + "build Phase 0"
```

Or manually:

1. Create the repo with the layout above
2. Create empty `VOLVE.md` with the template, fill in top sections
3. `claude login` on the VM, verify `claude -p "hello"` works
4. Write the morning digest prompt, test it interactively on the VM
5. Wire the systemd timer + Slack incoming webhook
6. Ship Phase 0, live with it for a week, then decide whether premise holds

---

Appendix: why not pi for the runtime

For the record, to make the decision inspectable later:

- Pi's Opus 4.7 access is via GitHub Copilot. Using Copilot as a daemon backend for non-IDE work is grey-zone TOS. Claude Code on a Volve Team seat is the supported path.
- Pi has no native MCP. Rebuilding Gmail/Calendar/Slack/GitHub as `extensions/*.ts` is work with no upside, since mature MCP servers exist for all of them.

- Pi's skill model is excellent for *interactive* workflows (`sv`, `solve-ticket`). It's not the right model for headless scheduled invocations — those want a simple prompt + context + output, which `claude -p` does natively.
- Pi stays the coding agent. Mac-side pi builds and maintains this project; Claude Code on the VM runs it. Clean separation.